Decision-Driven Evidence Disclosure for Collaborative Security Correlation

Ananay Srivastava, Jitendra Singh, Manik Bhardwaj, Ashish Jain, Mukunj Singh

Department of Computer Science and Engineering, Sharda University, India

***Abstract*—Cross-organization detection can require corroboration from several local views, yet routine export of logs creates disclosure and operational costs. This paper presents a finite evidence protocol in which each organization retains raw events, emits a versioned signal, and decides whether to release one requested categorical aggregate. A coordinator requests the least costly untried predicate that can still affect an unresolved rule, counts independent provenance groups, and records supported, contradicted, or unresolved outcomes. An organization-local ledger commits each released representation and its charge atomically; replay of the same snapshot, recipient, and catalogue entry cannot produce another fine-grained release. We implement the protocol with a durable correlation worker and compare five sharing strategies over 1,000 deterministic protocol cases: five scenarios, ten seeds, four policy conditions, and five strategies. In the no-fault slice, decision-driven requests retained support for all 20 attack-labelled cases and rejected the 20 benign cases incorrectly supported by minimal sharing. Relative to fixed richer sharing, they used 540 rather than 660 policy units and sent 200,100 rather than 230,460 payload bytes. Denial, coarsening, and budget faults yielded unresolved cases rather than unsupported positive decisions. These are synthetic protocol measurements, not a field detection result or a formal privacy guarantee. A planned CIC-IDS2017 replay remains incomplete because the official archive endpoint returned a registration page.**

***Index Terms*—collaborative intrusion detection, selective disclosure, evidence provenance, policy accounting, security correlation**

I. Introduction

An organization can observe an authentication burst without knowing whether peers see the same pattern. A coordinator can improve the context of that alert by asking for independent observations. However, a request for raw login records would expose identities, timings, and internal topology that the partner may have no reason to disclose. Even a derived count can reveal a rare event. The design question is therefore about which evidence to request, when to stop, and how to account for repeated requests. NIST guidance frames participation, sharing rules, and information handling as explicit decisions rather than automatic publication [1]. Existing platforms make threat intelligence exchange practical [2]; controlled sharing work shows that an organization can choose how much to contribute [3]. Our narrower target is an incident-specific, executable disclosure protocol for finite alert predicates.

Consider an authentication anomaly. The initial signal says that a local detector saw a high-scoring authentication burst; it does not distinguish credential rejection from a provider outage. A coordinator may first ask for the coarse failure-rate category, then for failure kind, and stop when an outage alternative is established. A fixed rich profile would continue releasing the remaining available fields. In the implemented protocol, each response is either approved, coarsened to `unknown`, or denied. A refusal is a valid observation about policy, never evidence for the attack hypothesis. This matters because treating missing evidence as confirmation can amplify correlated false alarms.

We make three contributions. First, we specify a bounded, decision-driven request rule over a declared evidence catalogue and explicit terminal outcomes. Second, we implement local transaction-based release accounting, replay-safe responses, and provenance-aware correlation. Third, we evaluate the sharing and outcome trade-off under controlled faults with frozen cases and published output files. The contribution is a combination of semantics and implementation measurements. It does not claim a new anomaly detector, anonymous sharing, differential privacy, or real-organization detection accuracy.

II. Related Work and Boundary

MISP supports collaborative exchange of structured threat intelligence, including indicators and contextual events [2]. STIX defines a representation for threat and observable information [4]; neither specifies this manuscript's incident-specific request policy. Earlier security-alert correlation sanitized contributed records [5], while SEPIA aggregated multi-domain events with secure computation [6]. Recent over-threshold private set intersection supports cross-network identification without revealing nonmatching addresses [7]. Those methods offer different privacy guarantees from our policy-limited categorical releases; we do not claim equivalent protection. Freudiger *et al.* study controlled sharing for predictive blacklisting [3], and Dara and Muralidhara survey privacy-preserving collaborative detection architectures [8]. Incremental disclosure has also been evaluated for human record linkage [9], establishing that on-demand release itself is prior art. Our narrower contribution is an implemented, finite security-predicate request sequence with local cumulative accounting, provenance-aware decisions, and explicit unresolved outcomes.

The utility of a sharing community cannot be assumed from the number of contributors: a large empirical threat-intelligence study found substantial aggregation of existing sources rather than uniformly new observations [10]. We therefore count enrolled provenance groups rather than message

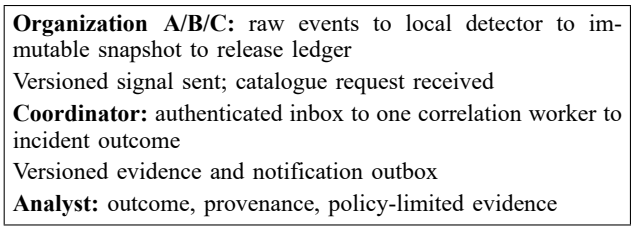


Fig. 1. Raw events stay in each organization. Only typed signals and approved catalogue responses cross the boundary.

volume and report unresolved outcomes alongside supported ones. This remains a protocol-level synthetic evaluation, not a claim that the groups observed independently operated networks.

Traditional intrusion-detection benchmarks address classifier performance under labelled traffic. CIC-IDS2017 was designed as a labelled network dataset with benign and attack periods [11], [12]. A replay against it could test detector and correlation quality, but our reported cases are generated by a local simulator, so benchmark accuracy cannot be inferred. Privacy claims also need care: Dwork and Roth formalize a guarantee in terms of neighbouring datasets and randomized mechanisms [13]. The integer units used in our ledger are policy costs set by an operator; they are not an ϵ value and do not establish that guarantee.

III. System and Threat Model

The system has n enrolled organization agents, one coordinator API, a durable correlation worker, and analyst clients. Each agent generates raw events and derived features locally. It is the authority for its snapshots and release policy. The coordinator receives a typed signal and may issue a request only for an entry in the finite catalogue. The authenticated recipient is `coordinator`; arbitrary SQL, log-file paths, and free-form queries have no wire representation. Figure 1 shows the trust boundary.

The adversary may submit repeated requests, delay or duplicate network delivery, correlate responses across time, or inject hostile content into local events. We assume enrolled keys are managed correctly, database storage on each agent is available during a transaction, and at most one representative vote is counted per enrolled provenance group. A malicious or compromised enrolled organization can still lie about a derived aggregate; the protocol does not attest to its underlying logs. Collusion between multiple nominally independent groups is outside the proven boundary. Correlation rules measure agreement with declared predicates, not the truth of a compromise.

A signal contains a unique signal and snapshot identifier, UTC observation interval, family, detector version and state, confidence or anomaly score, policy version, and recipient. The implementation limits observations to 1–600 seconds, accepts only supported schema versions, and rejects an unknown family before creating a correlation candidate. An incident groups signals by family and time bucket. The default bucket is 120 seconds, score threshold is 0.80, quorum is three independent groups, and request deadline is 90 seconds. These are configurable operational defaults, not tuned research parameters.

IV. Catalogue and Correlation Rules

The catalogue is intentionally small. Authentication has `auth_failure` (low/high/unknown; cost 1), `source_spread` (narrow/wide/unknown; cost 2), and `failure_kind` (rejected/unavailable/mixed/unknown; cost 2). Traffic has `request_rate` (normal/elevated/spike/unknown; cost 1), `source_spread`, and `service_errors` (low/high/unknown; cost 1). A coarsened response has value `unknown` and cost zero in the tested policy. The initial signal costs one unit. Values are enumerated, so a coordinator cannot request a raw IP address or an arbitrary finer range by varying a query string.

For a source o and family f, let V_o be its currently active released values. Its rule result is $d_f(V_o) \in \{S, C, U\}$ for support, contradiction, or unresolved. For authentication, S requires high failures, wide source spread, and rejected failures. Unavailable failures, low authentication failures, or narrow spread yield C. For traffic, S requires spike rate, wide spread, and high service errors; normal rate or low service errors yield C. Every other partial or unknown combination yields U. The short-circuit alternatives intentionally allow a decision without all three entries; they are rule semantics, not a probabilistic classifier.

An organization may submit multiple signals, but only one representative participates in an incident. Votes are then grouped by enrolled provenance identifier. If reports within a group disagree, its vote is unresolved. Given quorum q, the worker records S if at least q independent groups support, C if at least q contradict, and U otherwise; simultaneous support and contradiction quorums yield U with a conflict reason. The incident also records counts, pending requests, and coverage. Analyst workflow status (open, investigating, resolved) is separate from this evidential outcome. This prevents a case being marked resolved by a human from masquerading as proven technical support.

V. Decision-Driven Disclosure Protocol

The worker requests more evidence only while the incident is unresolved, has sufficient independent groups, and has not passed its deadline. For each unresolved participant it constructs the untried entries in the family catalogue. The implementation chooses

$$e^* = \operatorname*{arg\,min}_{e \in E_f \setminus (A_o \cup V_o)} \big(c(e), \mathrm{id}(e)\big), \tag{1}$$

where A_o is the attempted set, $c(e)$ is the declared unit cost, and the lexical identifier breaks ties. It asks for one entry and re-evaluates after the response. Selection is decision-driven in the limited sense that it runs only when the source rule remains unresolved and stops when the rule is decided. It is not an entropy optimizer or a proof of globally minimal disclosure. If every remaining predicate is blocked, the worker records a

reason such as `no_useful_authorized_evidence`; it never fabricates a support vote.

Each request includes schema version, request ID, snapshot ID, catalogue entry, recipient, and UTC deadline. The agent checks the current policy, snapshot presence and age, and available budget within a database transaction. An approval stores the exact response and its charge under $(\text{snapshot}, \text{recipient}, \text{entry})$. A retry for that tuple reuses the stored release and does not debit again. If policy has since tightened from fine to coarse, the old fine response is denied for further delivery. A denial carries no evidence value or identifier and has zero cost. Transaction failure rolls back both response and debit.

There are two hard limits: a per-snapshot cap and an organization-local rolling-hour cap. Consequently, creating more request IDs for one snapshot does not gain new values, while opening new snapshots still consumes the rolling cap. The initial signal is accounted before entering the durable agent outbox. The outbox retries transient failures with backoff and stable jitter; permanent rejection increments a visible gap counter. These rules handle ordinary retries and bounded release, but they do not prevent an attacker who controls many enrolled organizations or arbitrarily many time-separated snapshots from gathering a large cumulative view. A deployment must govern enrollment, retention, and policy epochs as well as numerical caps.

VI. Reference Implementation

CCE studies security-event correlation on serverless functions [14]. Our implementation instead keeps one database-backed worker owner; the choice simplifies ordering and restart recovery but does not establish serverless elasticity or high availability. The implementation uses a FastAPI API, PostgreSQL incident and evidence tables, a single worker that owns correlation transitions, and SQLite WAL ledgers in organization agents. The API validates typed Pydantic contracts and authenticates node keys before accepting signals or evidence. Unique identifiers make inbox processing and participation idempotent. The worker processes bounded batches, locks candidate incidents, updates their outcome, and writes notification records in the same database transaction. It also revisits pending incidents for deadline and evidence expiry, so an absence of new traffic does not leave an expired request active indefinitely.

Evidence records have a version and active flag. A correction or withdrawal changes the incident revision and can invalidate derived explanation state. The analyst view receives versioned notifications and can recover from a cursor if a bounded live queue overflows. PostgreSQL migrations and backup/restore were exercised locally; these checks establish that the tested schema can start and recover in that environment, not that a multi-site disaster recovery objective has been met. The correlation worker is deliberately singular in the reference deployment, limiting concurrent state ownership. It is a throughput and availability limit to revisit only with a measured need.

VII. Evaluation Design

We froze `research/cases.json` and its SHA-256 digest before recording outputs. The five scenarios are normal traffic, authentication attack, provider outage, traffic attack, and flash crowd. Each is run with ten seeds, three synthetic nodes, and the same 10-second event window. Attack ground truth consists of the two attack scenarios, giving 20 positive and 30 benign cases per strategy–fault slice. The local generator varies seeded event content but does not create independent real organizations; case counts are protocol repetitions, not a sample of deployments. Ground-truth labels are used by the evaluator and do not enter wire messages.

We compare (i) local only, which accepts the local high-score alert; (ii) fixed minimal, which transmits the initial signal but requests no aggregate; (iii) fixed richer, which attempts catalogue entries in fixed order; (iv) static escalation, which attempts that order in separate rounds; and (v) decision driven, which uses Eq. 1 and stops after a rule decision. Four conditions are run: no fault, policy denial of `failure_kind`/`service_errors`, coarsening of `source_spread`, and an exhausted per-snapshot budget of two units. The full factorial design is $5 \times 10 \times 5 \times 4 = 1{,}000$ cases. Each strategy–fault cell has 50 cases; the outcome calculation in the harness requires all three node decisions to agree, matching the configured quorum but without exercising arbitrary provenance conflicts.

Primary measurements are positive attack cases supported (TP), benign cases supported (FP), unresolved cases, policy units, released refinements, serialized payload bytes, and local protocol time. Payload bytes sum the encoded signal, request, and response bodies; they exclude TCP, TLS, database, and deployment overhead. Time is local wall-clock execution on one Windows CPU machine and excludes network transit. The disclosed integer cost is an accounting metric, not a privacy loss measure.

VIII. Results

Table I shows the no-fault slice. Local-only and fixed-minimal retain 20 true positives but also label 20 benign provider-outage or flash-crowd cases as supported. Richer evidence eliminates those false positives under the declared rules. The decision-driven strategy reaches the same 20 supported attack and zero supported benign cases as fixed richer and static escalation, while using 120 fewer units (18.2%), 60 fewer refinements (16.7%), and 30,360 fewer payload bytes (13.2%) than fixed richer. These are aggregate differences across the 50 fixed cases, not uncertainty-bounded estimates of population performance.

Table II makes the safety cost visible. Under denied evidence and exhausted budget, 40 of 50 decision-driven cases are unresolved; under coarsening, 20 are unresolved. In all three conditions its positive count is zero and its false-positive count is zero. “No false positives” here should not be read as good detection under faults: attack decisions are also withheld. A policy owner can therefore limit exposure, but analysts must

TABLE I
NO-FAULT SYNTHETIC SLICE; 50 CASES PER STRATEGY.

Strategy	TP	FP	Units	Releases	Bytes
Local only	20	20	0	0	0
Fixed minimal	20	20	120	0	48,180
Fixed richer	20	0	660	360	230,460
Static escalation	20	0	660	360	230,460
Decision driven	20	0	540	300	200,100

TABLE II
DECISION-DRIVEN STRATEGY UNDER INJECTED POLICY FAULTS.

Condition	TP	FP	U	Units	Bytes
None	20	0	0	540	200,100
Deny	0	0	40	240	164,640
Coarsen	0	0	20	420	200,340
Budget	0	0	40	240	164,640

see the unresolved workload rather than silently treating it as benign.

The no-fault mean round count is 2.0 for decision driven, 0.8 for fixed richer, and 2.4 for static escalation. This measure records the harness request schedule, not end-to-end user latency. The respective median local protocol times are 0.353, 0.391, and 0.379 seconds, but process scheduling and file-system variability were not controlled tightly enough to claim a latency improvement. The expected deployment trade-off is fewer disclosed bytes versus potentially more request-response round trips.

IX. WORKED TRACE AND COST DECOMPOSITION

The result is easier to interpret by tracing one source through each family. An authentication-attack window produces a high local score and an authentication-burst signal. The agent's private snapshot has `auth_failure=high`, `failure_kind=rejected`, and `source_spread=wide`. The coordinator's first request is `auth_failure`, which costs one unit. The source remains unresolved because high failure volume can be an outage. The next request is `failure_kind`, chosen before equally priced `source_spread` by lexical order. A rejected value still leaves the source unresolved until wide spread is known. The third approval completes the source's supporting rule. The initial signal plus three refinements cost $1+1+2+2=6$ units. All three independent synthetic groups must complete that path for the incident to be supported.

For a provider outage, the first request also returns high authentication failure, but the second returns `failure_kind=unavailable`. That value contradicts the attack rule immediately. The decision-driven strategy does not request `source_spread`; its per-source charge is $1+1+2=4$ units. Fixed richer still requests the third entry and reaches the same contradiction after spending six units. This example illustrates the paper's specific notion of decision value: an entry is worth requesting only while the declared rule needs it. No attempt is made to infer a private value from the owner refusing to release it.

The traffic family has an analogous path. An attack source reports `request_rate=spike`, `service_errors=high`, and `source_spread=wide`. The first two requests cost one unit each, and the final spread request costs two, giving five units with the initial signal. A flash crowd has spike rate but low service errors; it is contradicted after the second request at a total of three units. Fixed richer proceeds to the spread entry and pays five. In the ten normal cases, no high-scoring family candidate is issued, so neither the initial signal nor a refinement is charged. The no-fault totals follow directly from these four abnormal scenario paths, each repeated for ten seeds and three agents: decision driven uses $30(6+4+5+3)=540$ units; fixed richer uses $30(6+6+5+5)=660$. This arithmetic is a check on the summary CSV, not a claim that the chosen catalogue costs measure real information exposure.

TABLE III
NO-FAULT TRACE PER ACTIVE SYNTHETIC SOURCE. THE NORMAL SCENARIO EMITS NO CANDIDATE.

Scenario	Final rule	Decision units	Richer units
Authentication attack	Support	6	6
Provider outage	Contradict	4	6
Traffic attack	Support	5	5
Flash crowd	Contradict	3	5

This controlled decomposition also exposes the narrowness of the metric. A two-unit `source_spread` category might be less sensitive in one real organization than a one-unit `failure_kind` category in another. The costs are operator-specified labels and were held fixed only to make strategy comparison possible. The legitimate operational interpretation is that the same rule outcome was reached with fewer approved catalogue entries under this fixed policy. A privacy interpretation would require measuring inference risk about individual events or organizations under the actual query history. A bandwidth interpretation would require the full transport stack, which the byte metric intentionally omits.

X. CORRECTNESS PROPERTIES AND OPEN RISKS

The design treats inability to decide as a first-class result. An evidence request may be denied by policy, budget, expiry, or a missing snapshot. The response then has no evidence identifier and no value. Coarsening yields an explicit `unknown` category. Neither can satisfy a positive predicate. This is why denial and exhausted-budget faults suppress both false positives and true positives in Table II. The system still records the original local anomaly and the reason the cross-organization rule remained unresolved. An analyst can investigate locally, but the coordinator must not rewrite that case as benign merely because a partner protected its data.

Idempotency exists at several boundaries. A snapshot identifier can be queued once, and a conflicting reuse of that identifier is an error. An agent stores a released response by snapshot, recipient, and entry, so a fresh network request ID for the same data does not create a second release. A coordinator records signals in an inbox and processes a bounded batch through one worker. Participation is unique for the incident and organization. The release ledger and correlation database are distinct systems; there is no distributed transaction across them. If an owner commits a release but the coordinator never

receives the response, a retry retrieves the stored representation without another charge. If an incident times out before that retry arrives, the release has still happened and its cost remains in the owner ledger. That is an honest accounting of exposure even when it brought no analytic benefit.

The rule is deterministic but not necessarily robust to adversarially crafted data. An attacker who drives a legitimate source to show `failure_kind=unavailable` can force a contradiction in the authentication family, and an attacker who suppresses service errors can force a contradiction in the traffic family. The reference detector and categorical thresholds were not trained or validated against evasion. We therefore use *supported* and *contradicted* as names for rule states rather than "attack confirmed" and "safe." The analyst-facing evidence record should include each predicate and provenance group so that a responder can review the basis of a decision. A stronger deployment would add source integrity controls, calibration, and independent adversarial testing.

The worker also has an operational liveness limit. It creates one request per unresolved participant when there is no pending request, re-evaluates after responses, and stops at a deadline. A slow participant can leave the whole three-vote investigation unresolved. Increasing the deadline improves the chance of obtaining all responses but extends the time before a final outcome. Reducing it improves responsiveness but raises unresolved workload. The chosen 90-second default was not optimized in the synthetic harness, whose local protocol timings are far shorter. A deployment should measure response-delay distributions and choose a deadline based on analyst workflow needs. The correct timeout policy is a service-level decision, not a property of the greedy selection rule.

Finally, the trust boundary is not a secrecy proof. An approved category can reveal that an organization is observing abnormal traffic; a sequence of categories can reveal trends. The coordinator also sees which owner denied a request and when it did so. Authenticated transport and local ledgers restrict access and produce an audit trail, but they do not eliminate inference. We recommend treating these fields as sensitive operational data with access controls and retention limits. The reported budget cap is an engineering control for that surface and should be revised with a domain-specific risk assessment.

XI. Scenario-Level Measurement Check

Table IV breaks the aggregate byte result into the five frozen scenarios. There is no saving in either attack scenario because every positive predicate must be seen before support is possible under the declared rule. All savings occur in the two benign alternatives that can be contradicted early. For provider outage, decision driven sends 42,600 serialized bytes across ten cases versus 57,780 for fixed richer. For flash crowd it sends 42,300 versus 57,480. The two differences sum to the reported 30,360 bytes. This distribution matters: a deployment with few short-circuit benign cases would see much less saving, even if its total event volume were higher. Conversely, a catalogue with more expensive predicates following common benign alternatives might see more saving. Our result must not be extrapolated without a deployment's event mix and policy costs.

TABLE IV
No-fault serialized payload bytes by scenario, ten cases each.

Scenario	Decision	Richer	Saved
Normal	0	0	0
Authentication attack	57,690	57,690	0
Provider outage	42,600	57,780	15,180
Traffic attack	57,510	57,510	0
Flash crowd	42,300	57,480	15,180
Total	200,100	230,460	30,360

The same table also helps audit what the byte metric does and does not count. The initial signal is serialized once per active synthetic source. Each attempted refinement adds a request body and a response body, including denials and coarsenings. Byte counts vary slightly between semantically similar cases because identifiers and JSON values have different lengths. The measurement does not count raw event size, since raw events never cross the organization boundary in this protocol. It also does not include HTTP headers, TLS records, retries, or notifications sent to an analyst. The saving from omitting a refinement is therefore a saving in the application message bodies generated by this particular harness. Operational bandwidth should be remeasured at the transport boundary.

The synthetic cases were designed to test an ambiguity that a local high-score alert cannot settle. In a provider outage, a failed login may return a server error rather than a credential rejection. In a flash crowd, request volume rises without the service-error condition required by the traffic rule. The local-only and fixed-minimal strategies have no distinguishing evidence, so the harness labels the high-score anomaly as supported; this creates 20 false positives relative to the generator labels. The richer strategies ask enough questions to reject those benign alternatives. A different anomaly detector might distinguish them from local data, reducing the value of collaboration. The experiment holds the detector constant because it asks about the disclosure protocol; it does not prove that collaboration is better than improving local detection.

Two kinds of validity are distinct here. *Internal protocol validity* asks whether the code obeys the specified catalogue, budget, idempotence, and stopping rules. The frozen case manifest, direct CSV rows, sum checks, and recovery tests provide evidence for that claim. *External detection validity* asks whether the chosen predicates, costs, thresholds, and provenance assumptions generalize to realistic network traffic and organizations. This study cannot answer that question. The 50 cases in one no-fault slice are five authored scenarios under ten seeds, and the same generator supplies all three synthetic nodes. Reporting a conventional confidence interval over those 50 rows would overstate independence. We therefore report exact finite-case totals and mark wider conclusions as hypotheses for later study.

The failure matrix is similarly a protocol stress test. Denial is injected for a specified pair of entries; budget exhaustion

is implemented by reducing the per-snapshot cap; coarsening forces one entry to `unknown`. These are controlled interventions, not observed rates of real policy actions. The outcome semantics can be checked under them: a denied or unknown predicate does not become positive evidence, and a case unable to meet a rule remains unresolved. But the intervention cannot show how human analysts handle those cases, whether a partner would refuse more often for specific incidents, or how much information the coordinator could infer from refusal itself. Those require a field or participant study with explicit consent and access governance.

XII. Operational Semantics and Invariants

The implementation can be described as a transition system. Let I be an incident state, S_o an organization snapshot, $R_{o,e}$ a request, and L_o the organization-local release ledger. The coordinator creates a request only if I is unresolved, its deadline is live, source o has no pending request, and e belongs to the family's catalogue. The owner responds with one of $\{\text{approved}, \text{coarsened}, \text{denied}\}$. The worker validates that the response matches the pending request, persists any evidence as a new version, and re-evaluates I. A terminal outcome cancels outstanding requests. A denied or expired request prevents further refinement for that source in the current investigation. This is intentionally conservative: a later policy change needs a new investigation or an explicit restart operation rather than silent indefinite probing.

The release invariant is that, for any snapshot s, recipient r, and entry e, at most one stored representation exists: $|L_o[s, r, e]| \leq 1$. The charge invariant is $\text{charge}(s, r, e) = 0$ for a denied or coarsened response and the catalogue cost for an approved response. The transaction writes both response and charge, so a crash before commit exposes neither, and a crash after commit can replay the exact representation. The per-snapshot and rolling-hour inequalities are

$$\sum_{x \in L_o(s)} c_x \leq B_s, \qquad \sum_{x \in L_o[t-3600, t]} c_x \leq B_h. \tag{2}$$

The second sum spans snapshots owned by one agent. It is neither a distributed global budget nor an ϵ composition theorem. It makes repeated local requests visible and bounded within one window. Responses are immutable at their release version; a tightened policy refuses reuse of an earlier fine response, while already received information cannot be retroactively erased from a recipient.

The outcome invariant is that only independent provenance groups contribute votes, and no `unknown`, denial, timeout, missing snapshot, or absent response can count as support. A single organization's multiple signals remain one participation for the incident. Conflicting votes within a provenance group reduce that group to unresolved. If both supporting and contradicting quorum conditions hold, the incident is unresolved with a conflict reason. This conservative combination protects against several straightforward counting errors, although a forged provenance assignment or colluding organizations can defeat independence assumptions.

TABLE V
Protocol transitions and durable effects.

Event	Guard	Durable effect
Signal	Valid schema, enrolled node, score threshold	Inbox row and one organization participation
Request	Unresolved, live, no pending source request	One typed catalogue request
Approve	Policy and both caps permit	Exact response and charge in one ledger transaction
Coarsen	Owner policy requires coarse value	`unknown` representation; zero refinement charge
Deny	Policy, missing snapshot, expiry, or budget	No evidence; source cannot be silently counted
Expire	Deadline or evidence TTL reached	Re-evaluated incident and versioned notification

A. Why the greedy choice is bounded but not optimal

Equation 1 makes a reproducible local choice and avoids an adaptive search over raw data. It can request a cheap entry that ultimately proves unnecessary. For example, an unavailable authentication failure kind alone contradicts the attack rule, but `auth_failure` has lower unit cost and is requested first. A clairvoyant planner could ask the decisive kind immediately, but it would need a prior on local values and could favor fields with higher disclosure risk. The current rule avoids speculative utility estimates. It is best described as a deterministic adaptive baseline with rule-based stopping, not an optimal active-learning policy.

B. Failure boundaries

Network retries can duplicate delivery; unique signal and request identities, stored response representations, and transactional worker state make repeated processing operationally safe. They do not solve every distributed failure. A partition that lasts past the deadline produces unresolved rather than an inferred benign result. An agent that loses its SQLite file may lose release history; backups and recovery need an operator procedure before restarting that identity. A coordinator can learn from approved aggregates even if it later deletes its database, so retention and access rules remain necessary. The design also lacks a cryptographic commitment from raw logs to aggregate values; fabricated aggregates must be addressed by governance or stronger attestation in a later system.

C. A bounded coordinator loop

Figure 2 makes the request sequence explicit. It is a bounded loop because the catalogue has three entries per family, each attempted entry is recorded, and the incident has a deadline. The worker does not keep asking a source after a denial or expiry. An approved but nondecisive response can lead to another request only after the previous request has been applied to the incident state. This matters for backpressure: an agent never needs to handle an unbounded fan-out of speculative queries for the same incident. At most one request per participant is pending at a time in the reference implementation.

An owner has its own bounded loop. It first parses the exact request schema and rejects unknown catalogue entries or recipients. It checks the deadline and current policy, then looks

For each incident revision:

1) Recompute active evidence, provenance votes, and outcome.
2) If supported or contradicted, cancel pending requests and publish the revision.
3) If deadline passed, publish unresolved with deadline reason.
4) For each unresolved source without a pending or denied request, choose the cheapest untried catalogue entry.
5) If no useful entry remains, publish unresolved with coverage and reason.
6) Persist one request; on response or expiry, start again at step 1.

Fig. 2. Coordinator transition loop. Every transition is persisted before a notification is offered.

up the immutable snapshot. If a release for the tuple already exists, it returns the stored response unless the new policy forbids repeating the earlier fine value. Otherwise, it computes the permitted representation and prospective charge, checks the snapshot and rolling caps, inserts the response and charge, and commits. Network I/O occurs after the transaction. This ordering is essential: sending a response before committing a charge would permit an unrecorded release after a crash; charging before storing a response would create a debit with no replayable representation. The SQLite transaction is local to the owner and can therefore enforce the invariant without a distributed commit protocol.

The coordinator cannot prove that an owner actually enforced its ledger. It can validate the response contract, policy version, request linkage, and evidence identifier, but a compromised agent could lie about its local accounting or values. This is a distinct limitation from coordinator misuse. A stronger trust model might require remote attestation, signed release receipts, or an independently audited hardware module. Those mechanisms add deployment complexity and are not implemented here. The design instead makes the owner boundary clear and allows an organization to examine its own durable ledger. A complete security analysis should test key rotation, compromised enrolment credentials, replay outside the incident window, and malicious provenance claims.

The release tuple is coarser than the network request ID. Two coordinators asking for the same entry against the same snapshot and recipient see the same committed representation, not two independently sampled answers. This simplifies both audit and retry semantics but prevents the owner from changing a value within one snapshot without issuing a new snapshot identity. That is appropriate when a snapshot is immutable. If source data must be corrected, a later revision should be represented explicitly and prior evidence should become inactive in the coordinator. Reusing a snapshot ID with different underlying values would make the ledger's idempotence guarantee ambiguous and is rejected by the implementation.

D. Policy changes during an investigation

An investigation may outlive a policy update. Suppose an organization has approved `auth_failure=high` and then tightens its policy before the coordinator asks for `failure_kind`. The second request is evaluated under the new policy version. If the new policy denies that entry, the response is a denial; previously approved evidence remains part of the coordinator's incident record until it expires or is explicitly withdrawn. The correlation rule cannot complete from the remaining values and records unresolved. This differs from pretending the earlier approval never happened. A correct audit must retain both facts: data was released under the old policy and later disclosure was refused under the new one.

Another case is a retry of an earlier approved entry after the policy now requires coarsening. The ledger refuses to resend the earlier fine value. It also does not synthesize a new coarse value under the same release tuple, because that would create two representations for one immutable snapshot entry and complicate the at-most-once interpretation. A future version could support explicit supersession with a monotone representation order, but then it would need to prove that a later response never reveals more than the earlier one and to account for both transmissions. The present rule is conservative and simple to audit. It sacrifices some liveness when policy changes mid-incident.

Budget exhaustion has a comparable race. Two requests for different entries can arrive concurrently. A read-then-write check outside a transaction could allow both to observe remaining capacity and jointly exceed the cap. The owner uses a write transaction around cap checking and release insertion, serializing those decisions. The first eligible request can commit; the second sees the updated sum and may be denied. This produces order dependence when the budget cannot cover both entries, but it does not overspend. The coordinator's one-pending-request-per-source rule makes that race rare in the reference flow; the ledger still enforces the cap if callers bypass that scheduling expectation or if old retries arrive late.

E. Evidence freshness and incident revisions

The worker's outcome is a function of currently active evidence, not an irreversible label. Evidence has an age limit and can be corrected or withdrawn. When an item becomes inactive, the worker recomputes source decisions and provenance votes, then increments the incident revision if the outcome, reason, or summary changes. A formerly supported incident may become unresolved when a required predicate expires. This is the safer direction for stale evidence: it does not preserve a positive conclusion whose basis is gone. A downstream explanation generated from the older revision must also be marked stale, which is handled by the separate explanation lifecycle described in Paper 2. The sharing protocol here does not rely on an LLM to determine the outcome.

Freshness checks themselves can fail operationally. If the worker is stopped, an expiry may not be reflected in the materialized incident immediately, even though the underlying evidence timestamp is old. The implementation revisits expired evidence when the worker runs again and updates the incident. A high-assurance reader should also check evidence validity at read time, or expose a "worker delayed" health

state, before acting on the last materialized outcome. The tested local deployment verifies recovery behavior but does not demonstrate a hard upper bound on stale exposure under prolonged worker outage. That bound would need monitoring, alerting, and an availability objective.

Provenance requires similar lifecycle discipline. A group identifier is meaningful only while it refers to an enrolled and active source. If an organization is disabled, its evidence should not continue contributing a fresh vote. The worker's query joins only active organizations and recomputes outcomes. This guards against one class of stale membership error. It does not establish that a group truly corresponds to an independent underlying sensor. A partner could relay the same upstream signal through several identities, or a shared cloud provider could make ostensibly separate organizations fail together. In such settings, quorum should be defined over documented failure domains, not just account IDs. Establishing those domains is a governance task that the protocol makes explicit but cannot automate.

F. A useful boundary on formal claims

The invariants above are operational properties of the implemented state transitions. They are checked by code paths and targeted tests; this manuscript does not provide a machine-checked proof over all interleavings. In particular, the owner ledger's transaction and the coordinator database's transaction are separate. A message can be lost between them. The system is designed so such loss spends the owner's budget and may leave the incident unresolved, rather than allowing an uncharged release or fabricating a decision. A formal model could express this as a safety property under asynchronous delivery with eventual retries, then verify liveness only under a bounded-delay assumption. The present evaluation exercises representative failures but stops short of that formal verification claim.

XIII. Fault Analysis and Reproducibility

The fault matrix is more informative than a single headline reduction. Under no fault, fixed richer, static escalation, and decision driven all resolve the two attack scenario families and benign alternatives according to the declared rules. Under the denial fault, the distinguishing `failure_kind` and `service_errors` entries cannot be released. The 40 abnormal cases remain unresolved for decision driven; the ten normal cases have no candidate and remain `none`. Under a two-unit snapshot cap, the initial signal consumes one unit and there is insufficient space for all required predicates. Again 40 abnormal cases are unresolved. Under source-spread coarsening, the unknown category prevents confirmation of the two attack scenarios while some benign alternatives still contradict without it, yielding 20 unresolved cases. These patterns are consequences of the fixed rules and policy, not estimates of how frequently faults occur in production.

The same 50 scenario–seed pairs are replayed for every strategy and fault. This pairing permits a direct within-harness comparison of totals but does not justify a confidence interval

TABLE VI
Three disclosure strategies under each fault; cells are TP/FP/U across 50 cases.

Fault	Richer	Static	Decision
None	20/0/0	20/0/0	20/0/0
Deny	0/0/40	0/0/40	0/0/40
Coarsen	0/0/20	0/0/20	0/0/20
Budget	0/0/40	0/0/40	0/0/40

over unrelated organizations. The synthetic generator creates 10-second windows with 30 requests/s for normal, 200 requests/s for authentication attack and provider outage, and 800 requests/s for traffic attack and flash crowd. Authentication failures are marked with 401 status in the attack and 503 status in the outage. Traffic attacks inject elevated 503 responses. The source spread threshold is ten distinct addresses. These settings deliberately create two tempting benign alternatives for a coarse alert. They are disclosed here so a reader can see that the evaluation is a protocol test shaped around the implemented rules.

The result files are `research/results/disclosure.csv` and `research/results/summary.json`. The executable harness is `research/evaluate_disclosure.py`; the case manifest and digest are `research/cases.json` and `research/cases.sha256`. The evaluator refuses to run if the manifest digest differs or the output CSV already exists, protecting an existing measurement from accidental overwrite. Each run creates separate organization ledger paths by fault, seed, scenario, and strategy. The summary script aggregates the recorded CSV rather than regenerating results. Tests cover API contracts, ledger behavior, durable recovery, and evidence lifecycle; integration checks also exercised migrations, backup/restore, frontend build, and a three-organization local demo. The paper's numerical claims should be rechecked against those files whenever the implementation changes.

A. Interpretation of cost and transport

The 18.2% policy-unit reduction and 13.2% serialized-byte reduction in the no-fault slice are different quantities. An approved `source_spread` costs two units but its JSON body need not be twice as large as another predicate. Conversely, a zero-unit coarse response still consumes bytes and can reveal that the owner was queried. The measured payload also counts each request, which explains why fewer releases do not imply an equal percentage byte reduction. The static escalation strategy has the same units and bytes as fixed richer but a different round count because its schedule sends requests sequentially. No bandwidth measurement over TLS or WAN was recorded. A deployment claim about transfer cost therefore needs a network trace, compression and batching policy, and realistic retransmission rates.

B. Sensitivity to correlation assumptions

The harness uses three nodes and a three-vote outcome. If quorum were reduced to two, one faulty or colluding pair could dominate; if increased above three, these cases could

TABLE VII
AGGREGATE POLICY UNITS PER 50-CASE FAULT SLICE.

Strategy	None	Deny	Coarsen	Budget
Local only	0	0	0	0
Fixed minimal	120	120	120	120
Fixed richer	660	480	420	240
Static escalation	660	480	420	240
Decision driven	540	240	420	240

never resolve. If the temporal bucket were wider, unrelated events could be combined; if narrower, concurrent observations might split. The manuscript reports configured defaults rather than claiming they are optimized. A deployment should measure clock skew, base rates, organization overlap, and correlated detector errors before selecting quorum or a time window. Provenance labels are crucial because three feeds forwarding one sensor are not three independent observations. The current tests verify deduplication logic, while empirical independence of real organizations remains unmeasured.

C. Accounting under the fault interventions

The cost matrix in Table VII separates a stopping-rule benefit from a policy constraint. Without faults, decision driven spends fewer units than fixed richer because it does not request the third predicate for two benign alternatives. Under denial, it spends half as many units as fixed richer because it stops at the first denied decisive entry for each affected source. The saved units in this condition have a different meaning from no-fault savings: the unresolved count rises to 40, so the coordinator has paid less partly because it could not finish. The coarsening condition gives the same total policy units for the three richer strategies, even though decision driven sends fewer bytes than static escalation. The budget condition binds before the full catalogue can be released, so all three richer strategies spend 240 units and leave 40 abnormal cases unresolved. These interventions demonstrate why a paper should report utility and exposure together.

The fixed-minimal line is constant because it sends only the initial signal, which is allowed in these synthetic policies. Its apparent robustness to faults is misleading: it also produces the same 20 false positives in every slice and never tests the distinguishing predicates. Local-only incurs no sharing units, but it makes the same 20 false-positive support decisions and provides no cross-organization corroboration. These baselines bound two extremes: no cross-boundary evidence and initial-signal-only sharing. They are useful controls, not realistic final incident adjudicators. An analyst could manually resolve their alerts with data that the harness does not model.

The denial policy is especially instructive. In authentication, the first cheap failure-rate category can be approved but the later failure-kind category is refused. In traffic, request rate is approved but service errors are refused. The coordinator learns that an anomaly exists and learns one additional category, yet it cannot determine support or contradiction for either family. A naive implementation might count "no contradicting response" as support. The explicit unresolved state prevents that inference. It also has a cost: the organization has already released some information without delivering a final classification. If denials are frequent, an owner might prefer policy-aware negotiation before any release, such as advertising which catalogue entries it permits. That would reduce wasted partial disclosure but reveal policy configuration itself. We did not implement or evaluate such negotiation.

Coarsening creates a different ambiguity. The owner sends an explicit `unknown` source-spread value, so the coordinator knows the field was handled, yet the value cannot satisfy wide or narrow. Attack cases need wide spread to support and remain unresolved. Some benign alternatives can still contradict through failure kind or service-error values, so the unresolved count is 20 rather than 40. A coarsened response is therefore more informative about process than a denial but can be equally nondecisive for a particular rule. The implemented cost assigns it zero units because no fine value is released; whether even the existence of the coarse response deserves a charge is a policy choice for a real deployment.

D. Reproducing a result without overstating it

The case manifest is checked by digest before the evaluator writes a CSV. The evaluator refuses an existing output path, requiring a deliberate new result directory or a conscious removal of a prior artifact. This prevents accidental silent replacement of numbers cited by the manuscripts. Each output row records its scenario, seed, strategy, fault, ground-truth attack flag, outcome, unit cost, refinement count, serialized bytes, rounds, and local elapsed time. A reader can independently group rows by strategy and fault and recompute every table in this section. The summary JSON is a convenience view, while the row-level CSV preserves diagnostics when a total changes.

Reproducibility is not the same as independence. The pseudorandom seeds fix event realizations within a generator. They do not sample five new scenario types or three unrelated organizations. The normal and abnormal rates and status-code mappings are explicit in the simulator. If a researcher changes the detector threshold, catalogue order, or source rule, the protocol totals may change even with the same seeds. The scientific object to reproduce is the complete combination of case manifest, code revision, environment, and row-level output. This is why the manuscript confines its quantitative claim to the tested implementation and does not claim a statistically significant privacy or detection improvement.

There is also a construct-validity issue in the term *false positive*. In this experiment it means a benign-labelled synthetic scenario received a `supported` protocol outcome. That is not necessarily a false security alert in operational terms: a flash crowd may deserve attention, and a provider outage is itself an incident. The term is used to compare the declared attack-labelled task. A deployment should report separate labels for malicious activity, service health, and analyst actionability. A single binary attack label cannot capture all three. Likewise, *true positive* here means the rule supports a scenario authored as an attack; it does not verify that the detector would identify an unseen attack in live traffic.

E. Potential adaptations

There are plausible next policies beyond the two fixed schedules and the greedy selector. An owner could provide a signed capability map describing entries it may release at each coarseness, allowing the coordinator to avoid doomed requests. A coordinator could use a prior over benign alternatives to select a more informative predicate first. Or peers could compute a joint threshold using cryptographic techniques, reducing explicit categorical release. Each option shifts a different trade-off: capability maps leak policy, learned priors may be brittle or biased, and cryptographic protocols add computational and key-management cost. The present method deliberately chooses a deterministic finite baseline before those complexities. It gives researchers a concrete reference point for comparing them rather than claiming to dominate all alternatives.

XIV. Limitations and Threats to Validity

The scenarios are hand-authored and use a simplified local detector. Their benign alternatives are known to the authors and may favor the rule. Ten seeds vary event realizations but do not create independent traffic populations. The outcome metrics count support under declared predicates, not verified compromise. The synthetic three-node harness evaluates the protocol functions and local ledgers; it is not a WAN deployment, a privacy attack study, or a measurement of multi-tenant service capacity. Network transit, TLS framing, and analyst decision quality were not measured. The no-fault reduction is therefore a reproducible local comparison within a finite catalogue, not a universal savings rate. Refusal and exhaustion substantially increase unresolved cases, showing that lower disclosure can have an operational cost. The official CIC-IDS2017 replay is uncompleted, and primary references should be revalidated against final venue requirements before submission.

XV. Conclusion

Decision-driven disclosure gives a coordinator a bounded way to ask only while a declared rule remains unresolved and gives the owner local authority over each answer. In the controlled no-fault evaluation it preserved the richer baseline's rule outcomes with lower policy cost and fewer serialized bytes; injected policy restrictions converted many cases to unresolved. This supports a concrete protocol and implementation result. Field detection, privacy leakage, and human usefulness require separate evaluation.

References

[1] C. S. Johnson, M. L. Badger, D. A. Waltermire, J. Snyder, and C. Skorupka, "Guide to Cyber Threat Information Sharing," NIST SP 800-150, 2016, doi: 10.6028/NIST.SP.800-150.

[2] C. Wagner, A. Dulaunoy, G. Wagener, and A. Iklody, "MISP: The Design and Implementation of a Collaborative Threat Intelligence Sharing Platform," in *Proc. ACM WISCS*, 2016, pp. 49–56, doi: 10.1145/2994539.2994542.

[3] J. Freudiger, E. De Cristofaro, and A. E. Brito, "Controlled Data Sharing for Collaborative Predictive Blacklisting," in *Proc. DIMVA*, LNCS vol. 9148, 2015, pp. 327–349, doi: 10.1007/978-3-319-20550-2_17.

[4] B. Jordan, R. Piazza, and T. Darley, Eds., "STIX Version 2.1," OASIS Standard, Jun. 2021. [Online]. Available: https://docs.oasis-open.org/cti/stix/v2.1/os/stix-v2.1-os.html

[5] P. Lincoln, P. Porras, and V. Shmatikov, "Privacy-Preserving Sharing and Correlation of Security Alerts," in *Proc. 13th USENIX Security Symp.*, 2004. [Online]. Available: https://www.usenix.org/conference/13th-usenix-security-symposium/privacy-preserving-sharing-and-correlation-security-alerts

[6] M. Burkhart, M. Strasser, D. Many, and X. Dimitropoulos, "SEPIA: Privacy-Preserving Aggregation of Multi-Domain Network Events and Statistics," in *Proc. 19th USENIX Security Symp.*, 2010. [Online]. Available: https://www.usenix.org/events/sec10/tech/full_papers/Burkhart.pdf

[7] O. E. Arpaci, R. Boutaba, and F. Kerschbaum, "Over-Threshold Multiparty Private Set Intersection for Collaborative Network Intrusion Detection," in *Proc. 23rd USENIX NSDI*, 2026, pp. 2211–2226. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/arpaci

[8] S. Dara and V. N. Muralidhara, "Privacy Preserving Architectures for Collaborative Intrusion Detection," arXiv:1602.02452, 2016, doi: 10.48550/arXiv.1602.02452.

[9] H.-C. Kum, E. D. Ragan, G. Ilangovan, M. Ramezani, Q. Li, and C. Schmit, "Enhancing Privacy through an Interactive On-demand Incremental Information Disclosure Interface: Applying Privacy-by-Design to Record Linkage," in *Proc. SOUPS*, 2019, pp. 175–189. [Online]. Available: https://www.usenix.org/conference/soups2019/presentation/kum

[10] X. Bouwman *et al.*, "Helping hands: Measuring the impact of a large threat intelligence sharing community," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 1149–1165. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/bouwman

[11] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. ICISSP*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.

[12] Canadian Institute for Cybersecurity, "Intrusion Detection Evaluation Dataset (CIC-IDS2017)," Univ. of New Brunswick. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html

[13] C. Dwork and A. Roth, "The Algorithmic Foundations of Differential Privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, nos. 3–4, pp. 211–407, 2014, doi: 10.1561/0400000042.

[14] A. Serckumecka, I. Medeiros, and A. Bessani, "CCE: A Cloud-Based SIEM Correlation Engine Built on Serverless Functions," in *Proc. IEEE SRDS*, 2025, pp. 95–105, doi: 10.1109/SRDS69199.2025.00022.